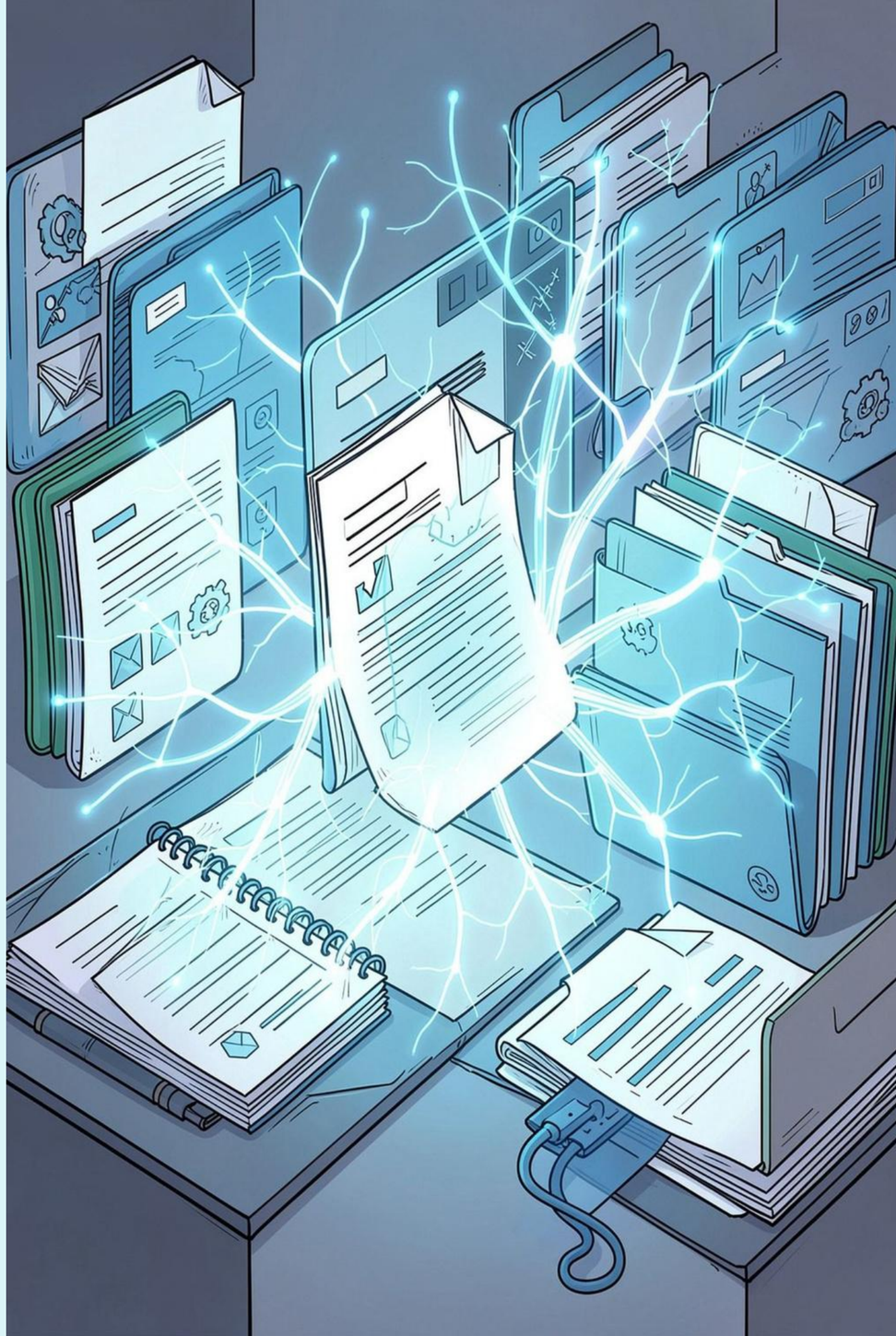
AI-Web Scraping and Intelligent Content Summarizer





Problem Statement

- Reading long documents is time-consuming.
- Key terms and obligations can be overlooked.
- Keyword search may miss intended meaning (semantic gap).
- Comparing versions is manual and error-prone.
- Information scattered across files, websites, and videos.

Our platform reduces these difficulties with AI-assisted analysis.



Project Introduction

- The project aims to make long and complex documents easier to understand.
- Accepts files, websites, and YouTube transcripts.
- Identifies different types of clauses.
- Finds information based on meaning.
- Answers questions using source information.
- Summarizes and compares documents.
- It highlights clauses that need attention for quick review.

Main message: Making complex documents easier to understand.

Project Objectives & Features



Multi-source Input

Read PDFs, DOCX, TXT, webpages, and transcripts.



Clause Extraction

Identify and categorise important clauses (privacy, liability, refunds).



Semantic Search

Ask natural-language questions to find relevant passages.



Summaries & RAG Answers

Generate concise summaries and source-backed answers (RAG).



Version Comparison

Compare document versions and highlight changes.



Dashboard

Central view: search, categories, summaries, and source links.

Example clause categories: Financial, Cancellation, Privacy, Liability, Termination, IP, Obligations.

Main Features — User View



- Upload document and automatic text extraction.
- View clause categories and attention levels (Low → Very High).
- Search with questions and get source-backed answers.
- Read concise summaries for quick understanding.
- Compare document versions with highlights.
- Supporting passages shown for transparency (not legal advice).

Attention levels are guides to focus review; they are not legal risk scores.

System Architecture



Keep the architecture simple: each block transforms input toward user-facing results.

Tools & Technologies Used

Area	Technologies
Frontend	React, Tailwind CSS, Recharts
Backend	Python, FastAPI
Machine Learning	PyTorch, Hugging Face Transformers
NLP	Sentence Transformers, spaCy, NLTK
Semantic Search	FAISS
File Processing	PyMuPDF, python-docx
Web & Video	Trafilatura, YouTube transcript API
Database	PostgreSQL, SQLAlchemy
Testing & Development	Git, GitHub, Postman, Pytest

How AI & Machine Learning Work

1. Text Extraction

Extract raw text from diverse file types and transcripts.

2. Chunking & Preprocessing

Clean text and split into smaller, meaningful sections.

3. Clause Classification

Fine-tune a transformer to label clauses (categories).

4. Embeddings & Search

Generate sentence embeddings; use FAISS for fast semantic retrieval.

5. RAG Answers & Summaries

Retrieve supporting passages and generate answers tied to sources.

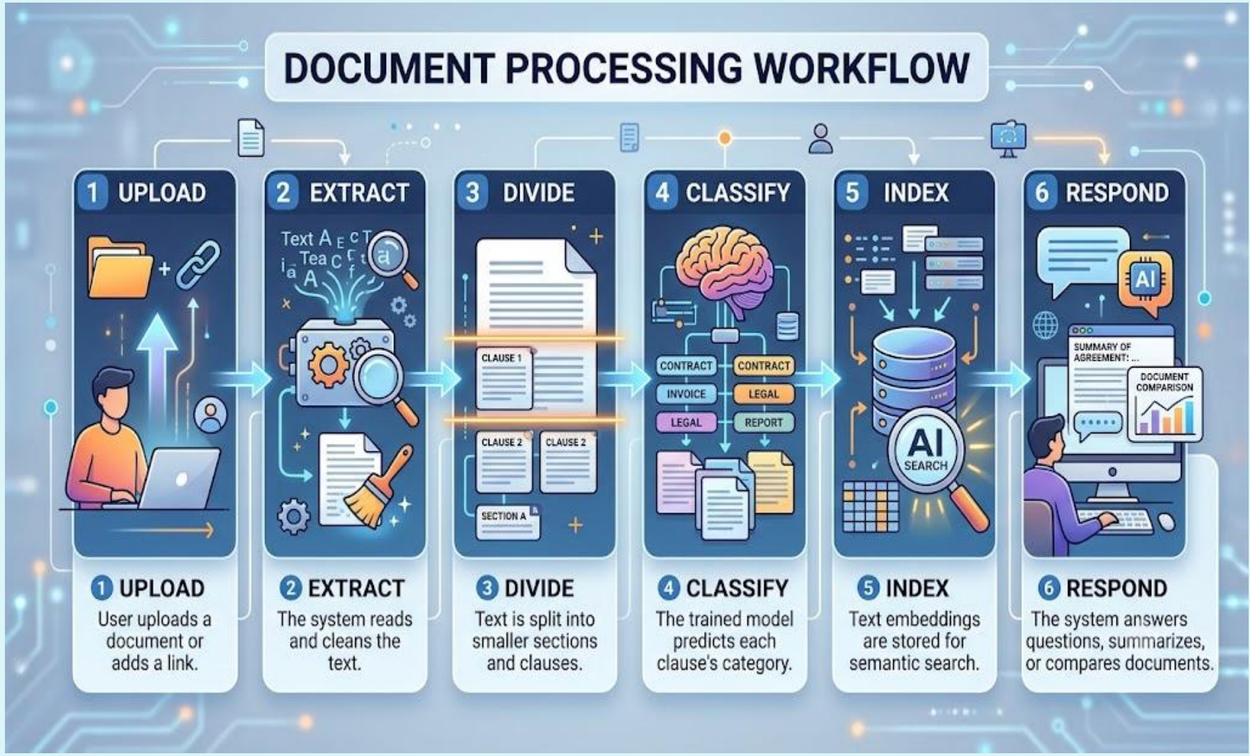
6. Testing

Validate models with a separate test set; check classification and retrieval quality.

Note: Do not assume final model performance without test results.

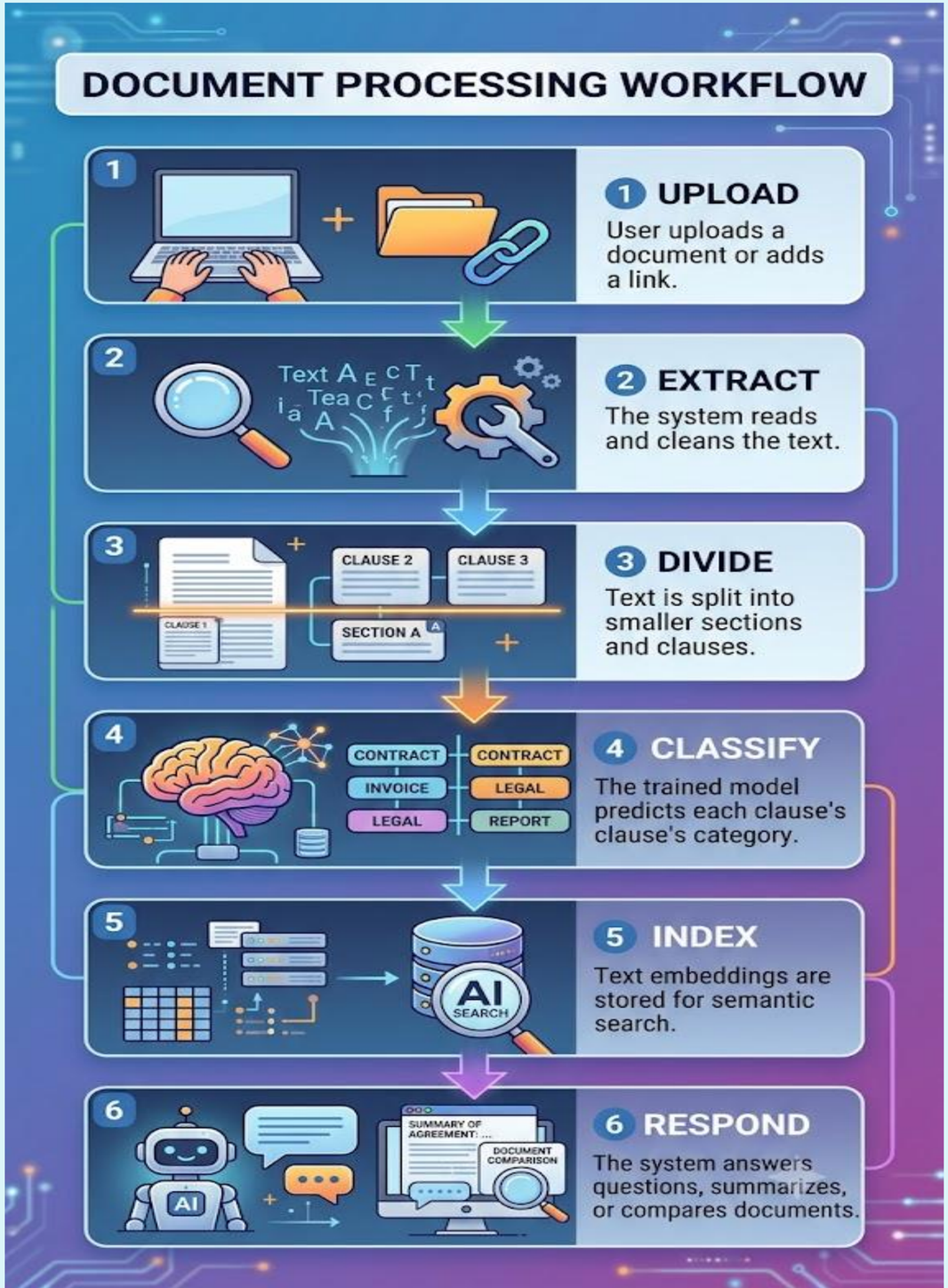
Working Process

- Upload: User uploads a document or adds a link.
- Extract: The system reads and cleans the text.
- Divide: Text is split into smaller sections and clauses.
- Classify: The trained model predicts each clause’s category.
- Index: Text embeddings are stored for semantic search.
- Respond: The system answers questions, summarizes, or compares documents.



Expected Outcomes & Evaluation

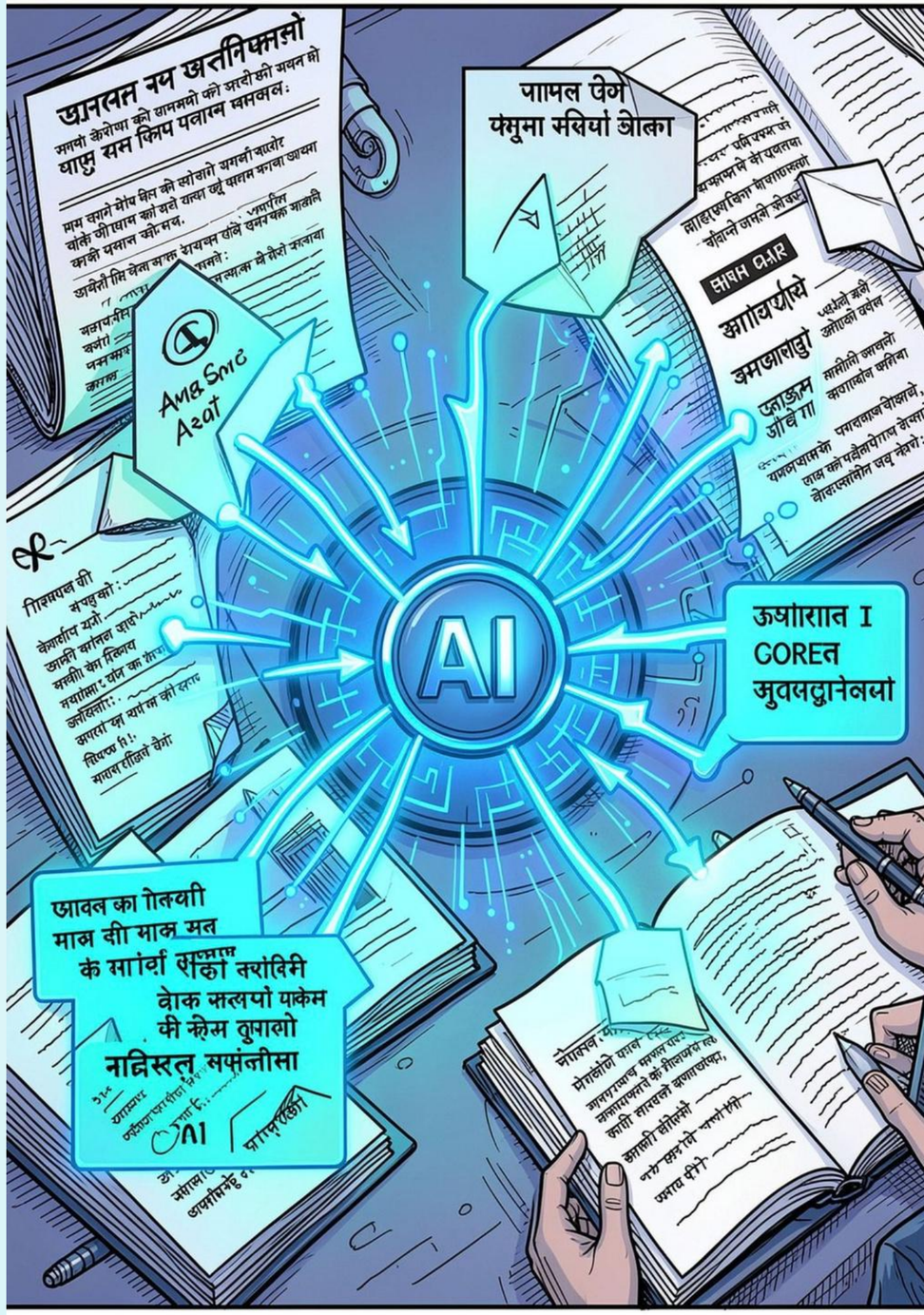
- Expected outcomes
- A single platform for different content types.
- Clause categories with confidence scores.
- Meaning-based search.
- Answers with references to source passages.
- Summaries and document comparisons.
- Easier identification of clauses needing attention.



Future Scope — Smarter AI Features

- Add Hindi and other Indian language support (OCR and NLP pipelines).
- Handle multilingual documents and mixed-language sections.
- Improve clause understanding for complex legal language.
- Detect relationships between clauses across a document.
- Auto-explain why a clause was flagged (rationale / evidence).
- Refine summaries and answers via iterative testing and user feedback.

These are planned improvements — labelled as future work.



Thank You